

计算机系统结构 (CS 2305)

班级号 _____ 学号 _____ 姓名 _____
课程名称 计算机系统结构 成绩 _____

一、选择题 (15 分)

1. 给定一个处理器有一个七阶段的流水线,流水段分别为: F D0 D1 I X0 X1 W
读寄存器发生在 I 段(issue 段),分支是否转移要在X0段才确定,如果流水线是单发射的,每个时钟周期只发射一条指令,那么当执行条件转移指令转移预测错误时,需要废除多少条已经进入流水线的指令?(假设进入流水线的第一条指令是条件转移指令)

(C)

- A. 2条 B. 3条 C. 4条 D. 5条

2. 关于下面程序段中的局部性(locality),哪些描述是正确的? (B)

```
sum = 0;
```

```
for (i = 0; i < n; i++)
```

```
    sum += a[i];
```

```
return sum;
```

- ① Sum变量的访问呈现了时间局部性
- ② 数组元素的访问呈现了空间局部性
- ③ 循环语句的执行呈现了时间局部性
- ④ 语句的顺序执行呈现了空间局部性
- ⑤ 函数的返回语句return 执行后,会破坏栈空间的局部性

- A、都对 B、①②③④ C、①②③ D、②③④

3. 以下对SIMD的描述,哪一个是不正确的? (D)

- A. SIMD 是一种数据级并行(data-level parallelism),可以用一条指令同时处理多个数据。
- B. Intel的 SIMD intrinsic 指令使用了字长比较长的寄存器,这样多个数据存放在一个寄存器中,对一个寄存器的访问等价于对多个数据的同时访问。
- C. 对控制流语句比较多的程序,例如:有较多的转移指令、if语句的程序,由于分支语句会对不同的数据进行不相同的操作,SIMD的性能可能会受到影响,
- D. SIMD是一种多线程处理器技术

4. 哪种技术可以在执行时提升以下代码的处理速度? (B)

```
loop: ADD R3 R4 R0
```

```
LD R4, 8(R4) # cache hit
```

```
BNEQZ R4, LOOP
```

- A. 带寄存器换名的乱序执行(out of order with register renaming)
- B. 转移预测(branch prediction)
- C. 多发射(multiple issue)

D. 多线程 (simultaneous multi-thread)

5. 定义了以下变量:

```
int x = random();
int y = random();
int z = random();
/* Convert to other forms */
unsigned ux = (unsigned) x;
unsigned uy = (unsigned) y;
```

从以下表达式中, 选出一项不永为真 (true) 的表达式: (D)

- A. $((x+y) \ll 2) + y - x == 5*y + 3*x$
- B. $x+y == ux+uy$
- C. $ux-uy == -(y-x)$
- D. $(x < y) == (-x > -y)$

6. 如果某计算机最复杂指令的执行需要完成5个子功能, 分别由功能部件A~E实现, 各功能部件所需时间分别为200ps、100ps、150ps、180ps和100ps, 采用流水线方式执行指令, 流水段寄存器延时为20ps, 则CPU时钟周期至少为: (A)

- A. 220ps B. 170ps C. 200ps D. 100ps

7. 以下关于超标量技术的叙述中, 错误的是 (D)

- A. 超标量技术开发的是指令级并行性
- B. 超标量方式执行指令时, 可以同时发射多条指令至流水线中
- C. 采用超标量技术的 CPU 必须配置多个不同的功能部件
- D. 超标量技术通过提高时钟主频, 加深流水线的段数, 以提高指令吞吐率。

8. 关于高速缓存, 以下的说法哪个不正确? (A)

- (a) 相同容量的全相联映射的高速缓存 (Fully associative caches) 比直接映射高速缓存访问延迟短。。
- (b) 相同容量的全相联映射的高速缓存缺失率比直接映射高速缓存 (direct-mapped caches) 缺失率 (miss rate) 低,
- (c) 相同容量的组相联映射 (set associative caches) 高速缓存在缺失率上比直接映射高速缓存低
- (d) 相同容量的组相联映射 (set associative caches) 高速缓存在访问延迟方面比全相联映射高速缓存低

9. 下面一个16 位的浮点数, 其中1位符号位 (sign), 5位阶码 (exponent), 10位尾数位 (fraction), 下列编码哪个表示 “负无穷大”? (D)

- A. 1 00000 1111111111
- B. 0 11111 1111111111
- C. 0 10000 0000000000
- D. 1 11111 0000000000

10. 以下是关于控制冒险的叙述:

1. 直接转移指令(无条件转移指令)不会导致发生控制冒险
2. 在分支转移指令后加入若干空操作指令可避免控制冒险
3. 采用转发(旁路)技术可以解决部分控制冒险
4. 通过编译器调整指令顺序,可解决一部分控制冒险,但不能解决所有控制冒险
5. 流水线段数的深度与控制冒险引发的开销无关

以上叙述中,正确的是(C)。

A、 1, 3, 5 B、 1, 3 C、 2, 3, 4 D、 2, 4 E、 全部

二、简答题(45分)

1. 以 IEEE 754 单精度浮点数格式表示十进制数-1.25, 以十六进制的形式表达。(5分)

答案: BF A0 00 00

2. 什么是精确中断? 如果有一条指令在执行阶段发生了两种类型的异常: 指令地址(取指)异常和 ALU 溢出, 那么哪一个异常会作为指令中断原因记录? 提示: 以五阶段流水线处理器作为流水线模型进行回答。(5分)

答案: 对于发生异常的单条指令, 中断被称为“精确”当且仅当满足

(1) 所有该指令之前的指令都已经提交其状态

(2) 所有后续指令(包括中断的指令)没有改变任何机器的状态

指令地址(取指)作为 指令中断原因记录

3. 什么是假共享(false sharing)? 什么时候会发生假共享?(5分)

参考答案:

缓存系统中是以缓存行或缓存块(cache line, cache block)为单位存储的。缓存行一般多于一个字(3分)。

当(共享变量的)多线程修改互相独立的变量时, 如果这些变量共享同一个缓存行, 就会因为高速缓存一致性协议(3分), 影响彼此的性能, 发生伪共享。

4. 某计算机的主存地址空间大小为 64KB, 按字节编址。高速缓存(Cache)采用 4 路组相联映射、LRU 替换算法和写回(write-back)策略, 能存放 4KB 数据。主存与缓存之间交换的主存块大小为 64B。

请问:

(1) 按字节编址, 主存地址有多少位? 如何划分? 要求说明每个字段的含义、位数。(2分)

(2) Cache 的总容量有多少位?(3分)

答案: 块大小 $64B=2^6 B$, Cache 行数 $4KB/64B=64$; Cache 4 路组相联, Cache 组数 $64/4=16=2^4$

主存大小 $64KB=2^{16} B$, 主存地址 16 位的划分分为: 6 位 tag cache 组号 4 位, 块内偏移量 6 位。

Write back: 1 位 dirty bit, LRU 替换: 每组有 4 行, 所以每行有: 2 位 LRU 位, 每行还有: 6 位 tag 标记位, 1 位有效位和 64B 数据。总容量 $64 * (6+1+1+2+64*8) = 33408$ 位

5. 简单描述一下快表 TLB (translation lookaside buffer)的作用是什么?

答案: TLB 是页表的高速缓存, 用于加快虚实地址的转换

- The page table for a process/program can be huge - and the entire page table will almost certainly not be cacheable.
- As a page table reference is required for every address translation, if every instruction and every data reference required a main memory lookup, performance would quickly and significantly degrade.
- The TLB is a fast cache for part of the page table - and if (a) the TLB contains 64-128 entries and (b) each page has $2^{14} - 2^{16}$ addressable entries, the seemingly small TLB can provide wide coverage

6. 一个程序中不可并行化部分为 25%, 可并行化部分占比 75%; 如果将处理器个数从 1 个增加到 5 个, 单个处理器的计算性能不变, 程序可以获得的加速比是多少?

答案: $1/(0.25 + 0.75/5)$

7. 假定机器 M 的时钟频率为 1GHz, 某程序 P 在机器 M 上的执行时间为 12 秒钟。对 P 优化时, 将其所有的乘以 2 的指令都换成了一条左移 1 位的指令, 得到优化后的程序 P'。已知在 M 上乘法规则的 CPI 为 4, 左移指令的 CPI 为 2, P 的执行时间是 P' 执行时间的 1.2 倍, 则 P 中有多少条乘法指令被替换成了左移指令被执行?

参考答案:

显然, P' 的执行时间为 10 秒, 因此, P 比 P' 多花了 2 秒钟, 因此, 执行时被换成左移指令的乘法指令的条数为 $1G \times 2 / (4 - 2) = 1G$ 条指令。

8. 设一个变量的值为 4097, 要求分别用 32 位补码整数和 IEEE 754 单精度浮点格式表示该变量 (结果用十六进制表示), 并说明哪段二进制序列在两种表示中完全相同, 为什么会相同?

参考答案:

$4097 = +1 \ 0000 \ 0000 \ 0001B = +1. \ 0000 \ 0000 \ 001 \times 2^{12}$

32 位 2-补码形式为: 0000 0000 0000 0000 0001 0000 0000 0001 (00001001H)

IEEE754 单精度格式为: 0 10001011 0000 0000 0001 0000 0000 000 (45800800H)

粗体部分为除隐藏位外的有效数字, 因此, 在两种表示中是相同的序列。

9. 假定一个虚拟存储系统的虚拟地址为 40 位, 物理地址为 36 位, 页大小为 16KB, 按字节编址。若页表中有有效位、存储保护位、修改位、使用位, 共占 4 位, 磁盘地址不在页表中, 则该存储系统中每个进程的页表大小为多少? 如果按计算出来的实际大小构建页表, 则会出现什么问题?

参考答案:

因为每页大小有 16KB, 所以虚拟页数为 $2^{40}B/16KB=2^{(40-14)}=2^{26}$ 页。

物理页面和虚拟页面大小相等, 所以物理页号的位数为 $36 - 14=22$ 位。

页表项位数为: 有效位+保护位+修改位+使用位+物理页号位数= $4+22=26$ 位。

为简化页表访问, 每项大小取 32 位。因此, 每个进程的页表大小为: $2^{26} \times 32b=256MB$ 。

如果按实际计算出的页表大小构建页表, 则页表过大而导致页表无法一次装入内存。

三、综合分析题（40 分）请在答题纸上答题，扫描上传附件至 Canvas。

1、（10 分）现在有一台机器，它的高速缓存容量为 1024-byte，采用直接映射，高速缓存数据块的大小为 16-byte。给定以下定义：

```
struct algae_position {  
    int x;  
    int y;  
};  
struct algae_position grid[16][16];  
int total_x = 0, total_y = 0;  
int i, j;
```

并假设：

- (1) sizeof(int) == 4; (2) grid 数组的内存起始地址为0
- (3) 高速缓存初始时空. (4) 所有的内存访问都是对grid数组的元素进行的.
- (5) 变量 i, j, total_x, and total_y 都存放在寄存器中

针对以下代码回答问题：

```
for (i = 0; i < 16; i++) {  
    for (j = 0; j < 16; j++) {  
        total_x += grid[i][j].x;  
    }  
}
```

```
for (i = 0; i < 16; i++) {  
    for (j = 0; j < 16; j++) {  
        total_y += grid[i][j].y;  
    }  
}
```

- A. 总共有多少次读内存操作？ 512.
- B. 多少次读操作引发了高速缓存失效（miss）？ 256 .
- C. 高速缓存的失效率（miss rate）是多少？ 50%.
- D. 针对高速缓存失效率，上面这段代码是否有可以改进的空间？如何改进？改进后高速缓存失效率为多少？

```
for (i = 0; i < 16; i++) {  
    for (j = 0; j < 16; j++) {  
        total_x += grid[i][j].x;  
        total_y += grid[i][j].y;  
    }  
}
```

Miss rate 减小到 128/512

2、（10 分）下图是一个实现了 RISC-V RV32I 指令集 的单周期处理器完整的数据通路，为了使得不同的指令在同一个处理器中完成不同的功能，控制器对指令译码后，要发出不同的控制信号。

表格中已经给出了部分指令应该具备的控制信号。其中 * 表示控制信号可以任意取值；BrUn 为 1 时，表示比较指令的两个比较值是无符号数； ImmSel 取不同的值，是因为不同的指令中的立即数在指令中不同的位置、以及立即数扩展的方法不同。

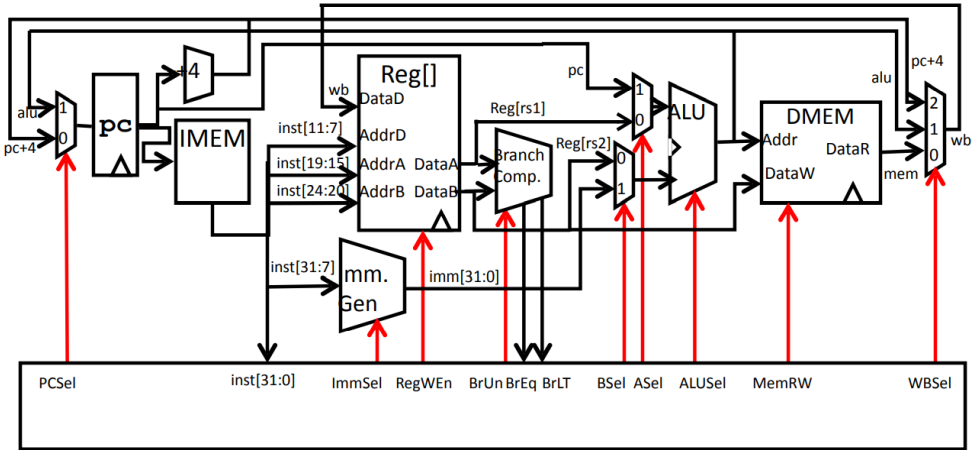


图. RV32I 单周期处理器数据通路

	BrEq	BrLT	PCSel	ImmSel	BrUn	ASel	BSel	ALUSel	MemRW	RegWEn	WBSel
add	*	*	0 (PC + 4)	*	*	0 (Reg)	0 (Reg)	add	0	1	1 (ALU)
ori	*	*	0	I	*	0 (Reg)	1 (Imm)	or	0	1	1 (ALU)
lw	*	*	0	I	*	0 (Reg)	1 (Imm)	add	0	1	0 (MEM)
sw	*	*	0	S	*	0 (Reg)	1 (Imm)	add	1	0	*
beq	1/0	*	1/0	SB	*	1 (PC)	1 (Imm)	add	0	0	*
jal	*	*	1 (ALU)	UJ	*	1 (PC)	1 (Imm)	add	0	1	2 (PC + 4)
bltu	*	1/0	1/0	SB	1	1 (PC)	1 (Imm)	add	0	0	*

表. 部分指令的控制信号

本题讨论 jalr rd, rs1, imm 指令。该指令的功能为：

$$R[rd] \leftarrow PC + 4$$

$$PC \leftarrow R[rs1] + imm$$

jalr 属于 I 型指令，I 型指令的编码格式如下：

31	20 19	15 14	12 11	7 6	0
imm[11:0]		rs1	funct3	rd	opcode
12		5	3	5	7
I-immediate[11:0]		src	ADDI/SLTI[U]	dest	OP-IMM
I-immediate[11:0]		src	ANDI/ORI/XORI	dest	OP-IMM

回答问题：为了支持 jalr 这条指令，各控制信号应该如何取值？

3、

(1) 找出下述指令之间的 RAW、WAW、WAR 数据相关性 (5 分)，
请将答案写在答题纸上。

- ① Loop: L.D F2, 0(R1) #读数据存入浮点寄存器 F2
- ② L.D F4, 0(R2) #读数据存入浮点寄存器 F14
- ③ MUL.D F6, F2, F4 #浮点乘法结果写入浮点寄存器 F6
- ④ SUB.D F4, F6, F10 #浮点减法结果写入浮点寄存器 F4
- ⑤ ADD.D F8, F8, F4 #浮点加法结果写入浮点寄存器 F8
- ⑥ ADDI R1, R1, #-8 #更新数组下标
- ⑦ ADDI R2, R2, #-8 #更新数组下标
- ⑧ BNEZ R1, LOOP #若数组下标不为 0, 跳转回 loop 继续循环语句

(2)

接下来, 我们考虑一个单发射、具有前向通路的按序流水线 (the fully-bypassed, single-issue, in-order pipeline)。如下图所示, 前向通路未画出。 并有以下假设:

浮点加法的结果要在 X1 流水段的结尾才能获得;

Load 指令读内存的结果要在 X3 流水段的结尾才能获得,

浮点乘法的结果要在 X4 流水段的结尾才能获得。

Store 指令要写入内存的数据要在 X2 流水段时准备好即可, 在 X2 流水段前不需要该数据。

分支指令的转移预测总是正确的 (没有因为 branch 指令导致的流水线停顿)

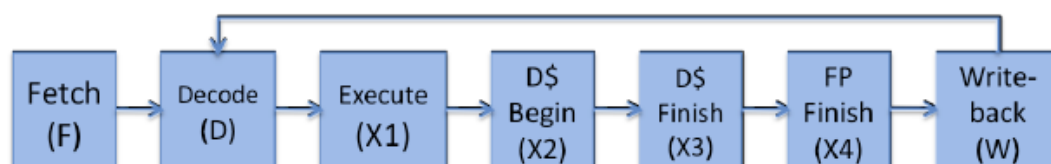


Figure 1. In-order pipeline. Full bypassing is not shown.

请在答题纸上, 画出上述八条指令一次循环的指令执行时空图。给出在不同周期时, 每一条指令所处于的流水段的名称 (用 F、D、X1、X2、X3、X4 和 W 表示各流水段)。 (6 分)

4、阅读以下这段RISC-V代码，回答问题：

- 1) 用C语言代码或者用文字描述本段代码的功能(5分)
- 2) 本段代码会输出一个整数，这个整数值为多少？(2分)
- 3) 代码中第8-10行、第18-20行的作用分别是什么？(2分)

解答：求调用求阶乘函数，计算5!； 结果为120； 参数n入栈、返回地址入栈； 参数n入栈、返回地址入栈； 参数出栈、返回地址出栈；

main:

```
1. addi a0, x0, 5
2. jal ra, FAC
3. addi a1, a0, 0
4. addi a0, x0, 1
5. ecall # Print Result
6. addi a0, x0, 10
7. ecall # Exit
```

FAC:

```
8. addi    sp, sp, -8      #adjust stack pointer
9. sw      ra, 4(sp)       #save ra (return address)
10. sw     a0, 0(sp)       #save a0 (argument)
11. addi    t0, x0, 1
12. bge     a0, t0, L1     #if ..., go to L1
13. addi    a0, x0, 1      #else ...
14. addi    sp, sp, 8      #adjust stack pointer
15. jr      ra            #return to caller (1st)
```

L1:

```
16. addi    a0, a0, -1     # decrement
17. jal     fa             #call fa with new argument
18. lw      t1, 0(sp)
19. lw      ra, 4(sp)
20. addi    sp, sp, 8
21. mul     a0, a0, t1
22. jr      ra            #return to caller (2nd)
```